

PDE-Agents-lite: An Agent-Orchestrated Finite Element Heat Solver with Verifiable Numerical Methods

CSE402 Numerical Analysis, Simulation and Modeling Project Report

Group 9 Section C

Sadman Sakib (2105121)
Md. Taskeen Alam Towfique (2105122)
Raiyan Siddiqui (2105130)
Mesbah Uddin Ahamed (2105139)
Mehedi Hasan Khan (2105141)

Abstract

Finite element heat simulations require consistent material data, boundary conditions, linear solves, and numerical checks. We developed PDE-Agents-lite, a lightweight adaptation of the PDE-Agents architecture that routes simulation, analytics, and run-history requests to specialist agents while exposing the numerical core for inspection. The system assembles linear triangular finite elements on rectangular meshes, handles Dirichlet, Neumann, Robin, and insulated boundaries, and advances transient problems with a configurable theta method. Handwritten Gaussian elimination and pivoted LU are available alongside a SciPy sparse reference solver. A graph of materials and previous runs supports property lookup and retrieval of similar configurations; SQLite records completed runs. We evaluated three analytical heat-equation benchmarks, solver agreement, an explicit-stability diagnostic, and a steady copper-plate case. In the saved five-mesh study, the constant-source problem has an L^2 convergence rate of 2.000 and the transient Fourier case has a fitted rate of 2.149, although the latter mixes spatial and temporal error. A 20×20 copper plate solved with custom LU matched its linear analytical temperature profile to a maximum nodal error of 2.27×10^{-12} K. These results support the numerical implementation on the tested cases. Retrieval quality and agent reliability have not yet been established by the base paper’s controlled ablation protocol.

Keywords

finite element method, heat equation, multi-agent systems, knowledge graph, numerical verification, LU decomposition

1 Introduction and Base Paper Review

Turning a natural-language heat-transfer request into a trustworthy simulation involves more than generating a temperature field. The system must identify the material, specify a well-posed problem, assemble the discretization, solve it, and interpret the result. A numerically plausible answer can still be wrong if a material property or boundary condition is guessed incorrectly.

Our base paper, *PDE-Agents: An LLM-Orchestrated Multi-Agent Framework for Automated Finite Element Simulations with Knowledge Graph-Augmented Reasoning*, combines a

LangGraph supervisor with Simulation, Analytics, and Database specialists, a DOLFINx/PETSc solver, and a Neo4j knowledge graph of materials, known issues, and prior runs [1]. It reports three analytical verification cases with approximately second-order spatial convergence, as well as a 50-task comparison of knowledge-graph retrieval modes. Its “KG Smart” mode injects similar past runs before agent reasoning and allows conditional retrieval. The paper reports a 100% success rate, mean physics score 0.933, and material-property fidelity 0.926 for that mode. These are *the base paper’s* results, not measurements of our implementation.

Our objective is to adapt that workflow for a numerical-analysis course and make its finite element calculations explicit. We replace the external numerical runner with directly implemented P1 assembly, custom Gaussian elimination and LU, and analytical verification. We use a local NetworkX graph and TF-IDF retrieval for lightweight material and run context, with SQLite persistence. The scope is two-dimensional constant-property heat conduction on rectangles; the system does not reproduce the base paper’s complex geometries or controlled agent ablation.

2 Methodology

2.1 Governing equation and finite element formulation

For temperature T on a rectangular domain Ω , the implemented model is

$$\rho c_p \frac{\partial T}{\partial t} - \nabla \cdot (k \nabla T) = Q, \quad (1)$$

where k is thermal conductivity, ρ density, c_p specific heat, and Q volumetric heating. A structured $N_x \times N_y$ grid is divided into two triangles per cell. With linear basis functions ϕ_i , the assembled semi-discrete system is

$$\begin{aligned} M\dot{\mathbf{T}} + K\mathbf{T} &= \mathbf{F}, \\ M_{ij} &= \int_{\Omega} \rho c_p \phi_i \phi_j d\Omega, \\ K_{ij} &= \int_{\Omega} k \nabla \phi_i \cdot \nabla \phi_j d\Omega. \end{aligned} \quad (2)$$

For a triangle of area A , the consistent local mass matrix is $\rho c_p A [2 \ 1 \ 1; 1 \ 2 \ 1; 1 \ 1 \ 2]/12$; the local stiffness uses the constant P1 gradients. Source terms and boundary fluxes enter $\mathbf{F}$. Dirichlet values are imposed by symmetric row and column elimination. Neumann flux adds edge loads; Robin convection adds both an edge matrix and an ambient-temperature load. An insulated edge contributes zero flux.

For the default Backward Euler integrator, a time step solves

$$\left(\frac{M}{\Delta t} + K\right) \mathbf{T}^{n+1} = \frac{M}{\Delta t} \mathbf{T}^n + \mathbf{F}. \quad (3)$$

The implementation also supports $\theta = 0$ (Forward Euler) and $\theta = 1/2$ (Crank–Nicolson). It shortens the last step when needed to reach the requested final time.

2.2 Linear solves and stability diagnostic

The educational solvers perform Gaussian elimination or factor $PA = LU$ with partial pivoting, followed by triangular substitution. For larger sparse systems, SciPy supplies the reference direct solve [8]. Dense custom methods require $O(n^2)$ storage and approximately $O(n^3)$ factorization work, so the complete refinement series uses the sparse solver; the custom methods are cross-checked on feasible meshes.

The code also contains a standalone power method for matrix eigenvalue exercises. The *runtime* explicit-step diagnostic uses SciPy’s generalized sparse eigensolver on the free degrees of freedom,

$$K_{ff} \mathbf{v} = \lambda M_{ff} \mathbf{v}, \quad \Delta t_{FE, \max} = \frac{2}{\lambda_{\max}}. \quad (4)$$

This limit diagnoses Forward Euler stability; it does not automatically select a step size. Backward Euler remains stable above this bound for the linear diffusion model, although a large step may reduce accuracy.

2.3 Agent and knowledge workflow

A LangGraph state graph [5] routes each request to Simulation, Analytics, or Database. The simulation tools retrieve materials, check rules, validate the configuration, run the solver, and record its result. In the live-model path, up to three similar past configurations are inserted into the specialist prompt before tool use. A deterministic offline path exercises routing and tool integration without a model call; it is a plumbing test, not evidence of autonomous reasoning. NetworkX [2] stores material and run nodes, TF-IDF/cosine similarity from scikit-learn [6] ranks past requests, and SQLite stores run records [7]. The seeded material values are project inputs rather than a separately validated materials dataset.

3 Implementation and Experiments

The numerical implementation is organized into mesh generation, assembly and boundary handling, linear solvers, the production heat solver, and analytical verification. NumPy arrays [3] store element and solution data; SciPy sparse matrices hold the assembled operators. The experiment scripts produce the figures with Matplotlib [4]. Table 1 records the principal configurations.

Table 1: Experiment configurations. Each $N \times N$ mesh has $(N + 1)^2$ nodal degrees of freedom.

Experiment	Configuration
Analytical V&V	Unit square; $N = 8, 16, 32, 64, 128$; six-point triangle quadrature; SciPy sparse solve for the full series.
Solver agreement	Steady constant-source case; $N = 16$ (289 nodes); SciPy, custom Gauss, custom LU.
Copper plate	$1 \text{ m} \times 1 \text{ m}$; $N_x = N_y = 20$ (441 nodes); $k = 385 \text{ W/(m K)}$; left/right 373/273 K; top/bottom insulated; custom LU.
Stability exam-ple	6×6 mesh; compare Forward and Backward Euler against the computed explicit step limit.

The three closed-form benchmarks are: (i) a steady linear profile $T = x$ with left/right Dirichlet values 0/1 and insulated top/bottom; (ii) a transient Fourier mode $T = e^{-2\pi^2 t} \sin(\pi x) \sin(\pi y)$ with zero temperature on all edges, $k = \rho = c_p = 1$, $\Delta t = 10^{-4}$, and $t = 10^{-2}$; and (iii) a steady constant-source case with $k = 1$, $Q = 1000$, zero-temperature left/right edges, insulated top/bottom, and exact solution $T = 500(x - x^2)$. These are synthetic analytical benchmarks, so no external dataset is used. For each case we calculate the integrated error

$$\|e\|_{L^2(\Omega)} = \left[\int_{\Omega} (T_h - T_{\text{exact}})^2 d\Omega \right]^{1/2} \quad (5)$$

using six-point triangular quadrature and fit the slope of $\log \|e\|$ against $\log h$, where $h = 1/N$. The base paper uses five mesh levels and a different quadrature rule; its tabulated Poisson problem also uses a different source normalization [1]. We therefore compare convergence *rates*, not raw error magnitudes.

4 Results and Discussion

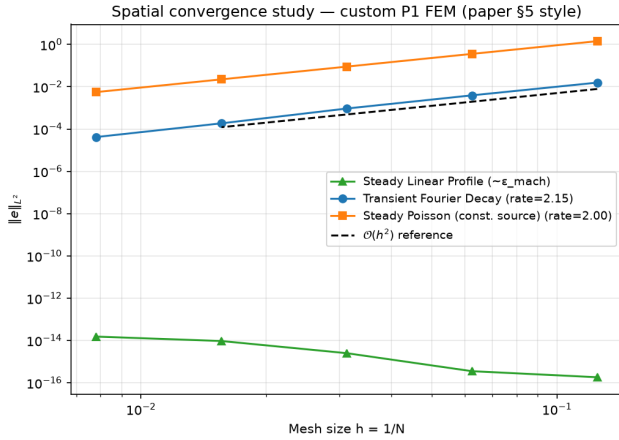
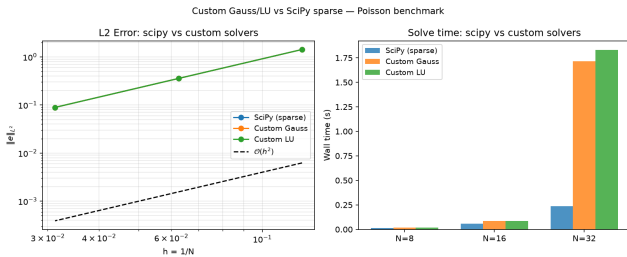
4.1 Analytical verification and solver agreement

Table 2 and Fig. 1 summarize the saved five-mesh study. The linear profile is representable in the P1 space and remains near floating-point precision; its fitted slope has no useful interpretation. Halving h reduces the Poisson error by four, yielding a rate of 2.000. The Fourier fit is 2.149, close to the expected P1 spatial order, but its fixed Δt leaves a temporal error component. The base paper reports rates 2.00 for its Poisson case and 2.04 for its Fourier case [1]; our different setup and temporal-error cancellation preclude a claim of exact replication.

At $N = 16$ in the Poisson benchmark, SciPy, custom Gauss, and custom LU all give $\|e\|_{L^2} \approx 0.3565902$; their reported errors differ by less than 9×10^{-14} . Figure 2 shows their overlapping accuracy curves and an illustrative timing comparison. Each timing bar is one wall-clock run that includes more than factorization, so it supports only a qualitative scaling comparison. A separate project audit reports 18/18

Table 2: Saved L^2 errors across five mesh levels. The fitted rate uses all five values.

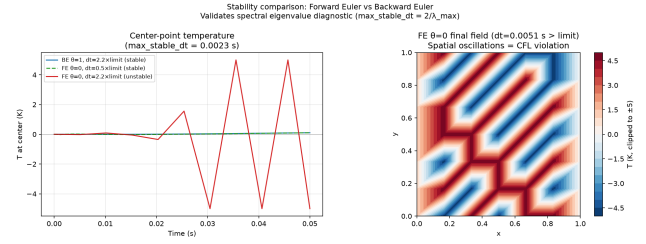
N	Linear	Fourier	Poisson
8	1.83e-16	1.56e-2	1.4264e0
16	3.53e-16	3.91e-3	3.5659e-1
32	2.51e-15	9.30e-4	8.9148e-2
64	9.42e-15	1.86e-4	2.2287e-2
128	1.53e-14	4.17e-5	5.5717e-3
Rate	exact	2.149	2.000

**Figure 1: Saved analytical mesh-refinement study. The linear case lies near roundoff; the Fourier and Poisson curves show approximately second-order behavior.****Figure 2: Accuracy agreement and illustrative cost for the three linear-solver choices. Timings are single measurements, with no confidence intervals.**

tests passing and 15/15 targeted probes passing; these are functional checks rather than an agent-quality study.

4.2 Copper-plate case study

We directly validated the requested 20×20 copper configuration, checked pre-run warnings, and solved with custom LU. Validation returned no errors and the rule engine returned no warnings. The numerical result had 441 nodal

**Figure 3: Forward/Backward Euler comparison around the generalized-eigenvalue explicit stability limit. The unstable curve is clipped for display in the saved figure.**

unknowns, a temperature range of 273–373 K, and a mean of 323 K. For this source-free homogeneous plate, the analytical solution is $T(x, y) = 373 - 100x$ K. The maximum nodal difference was 2.27×10^{-12} K. One measured solve took 0.269 s on the local environment; this single timing is not a hardware-independent performance claim. Since constant k cancels from this steady, source-free solution, the field does not establish the fidelity of the copper-property lookup. The project’s seeded copper triplet is $k = 385$ W/(mK), $\rho = 8960$ kg/m³, and $c_p = 385$ J/(kg K).

4.3 Stability and comparison with the base paper

In the saved 6×6 stability experiment, the computed Forward Euler limit is approximately 2.3×10^{-3} s. Figure 3 shows bounded Forward Euler behavior below the limit and an oscillatory growing field above it, while Backward Euler remains bounded. This verifies the diagnostic for the illustrated case; it does not show that the system automatically chooses a better time step.

The project reproduces selected verification ideas from the base paper and adapts its supervisor/specialist pattern. Its retrieval uses TF-IDF over local run descriptions instead of the paper’s embedding-indexed Neo4j GraphRAG; its offline agent uses deterministic parsing. Although a live-model route has a one-request smoke test, there is no frozen-task KG On/Off/Smart ablation, material-property-fidelity score, or repeated agent-success measurement for this project. The base paper’s success and physics scores are external baselines, not results of this project.

5 Conclusion and Future Work

PDE-Agents-lite provides an inspectable P1 heat-equation solver inside a three-specialist agent workflow. The analytical steady Poisson case exhibits the expected second-order L^2 convergence; custom Gauss and LU agree with the sparse reference on tested systems; and the 20-by-20 copper plate reproduces its exact linear field. The generalized-eigenvalue check distinguishes stable and unstable explicit steps in a demonstration.

The current implementation is limited to rectangles with uniform properties. Dense custom solves become expensive on large meshes, the Fourier refinement study still mixes time and space error, and the local material graph has no authoritative provenance for every seeded property. Future work includes isolating temporal error on refined meshes, benchmarking repeated solver timings, validating additional materials and boundary combinations, and running a controlled, frozen-task retrieval ablation with configuration fidelity, field error, tool cost, and numerical validity as separate metrics. A measured step-selection policy could then be compared with the present diagnostic-only behavior.

6 Code Submission

<https://github.com/Salixandom/pde-agent>

References

- [1] Sayan Adhikari, Gulshan Noorsumar, and Øyvind Jensen. 2026. PDE-Agents: An LLM-Orchestrated Multi-Agent Framework for Automated Finite Element Simulations with Knowledge Graph-Augmented Reasoning. arXiv:2606.07850 [physics.comp-ph] <https://arxiv.org/abs/2606.07850>
- [2] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the 7th Python in Science Conference*. 11–15. <https://networkx.org/documentation/stable/reference/citing.html>
- [3] Charles R. Harris et al. 2020. Array programming with NumPy. *Nature* 585 (2020), 357–362. doi:10.1038/s41586-020-2649-2
- [4] John D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science and Engineering* 9, 3 (2007), 90–95. doi:10.1109/MCSE.2007.55
- [5] LangChain. 2026. LangGraph Documentation. <https://langchain-ai.github.io/langgraph/> Accessed 2026-10-02.
- [6] Fabian Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [7] SQLite Project. 2026. SQLite Documentation. <https://www.sqlite.org/docs.html> Accessed 2026-10-02.
- [8] Pauli Virtanen et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. doi:10.1038/s41592-019-0686-2